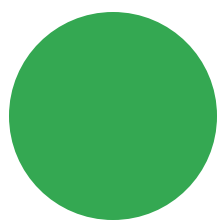




Core

concepts



Introduction

You learned state namespaces:

- `temp`: - Discarded after turn
- Session (no prefix) - Persists across conversation
- `user`: - Persists across sessions
- `app`: - Global application config

You learned state communication:

- `output_key` to write agent output to state
- `{key}` templating to read from state

Now we'll combine both: Using state namespaces effectively in multi-agent workflows.

Core concept: State flow in workflows

How state flows between agents

Every agent in a workflow shares the same session state:

```
Python
# Agent A writes to state
agent_a = LlmAgent(
    output_key='research_data', # Writes to state['research_data']
    instruction='Research the topic'
)
```



```
# Agent B reads from state
agent_b = LlmAgent(
    instruction='Write article using: {research_data}'
    # Reads state['research_data']
)

# Both share the same state
workflow = SequentialAgent(sub_agents=[agent_a, agent_b])
```

The key insight: All agents in a workflow hierarchy share one session state. Data written by one agent is immediately available to all others.

State flow patterns

Pattern 1: Linear flow (sequential)

None

Agent A → writes 'data_a' → Agent B reads 'data_a' → writes 'data_b' → Agent C reads 'data_b'

Pattern 2: Fan-out flow (parallel)

None

```

Agent A writes 'input' ─┐
                        │┐ Agent B writes 'result_b' ┐
                        │├──────────────────────────┤
                        │└┐ Agent C writes 'result_c' ┘┐
                        └─┘                               ┘┐ Agent D reads both

```

Pattern 3: Accumulating flow (loop)

None

Agent A writes 'draft' → Agent B writes 'feedback' → Agent A reads 'feedback', updates 'draft' → repeat

Core concept: Namespaces in workflows

Choosing the right namespace

Namespace	Use in workflows	Example
<code>temp:</code>	Intermediate signals (discarded after turn)	<code>temp:current_stage</code>
Session	Workflow results (persists in conversation)	<code>analysis_result</code>
<code>user:</code>	User preferences affecting workflow	<code>user:preferred_format</code>
<code>app:</code>	Global workflow configuration	<code>app:max_iterations</code>

Namespace examples in workflows

Session namespace (most common in workflows):

```
Python
# Workflow results persist across conversation
agent = LlmAgent(
    output_key='analysis_complete', # Session namespace by default
    instruction='Analyze the document'
)
```

Using user preferences in workflows:

```
Python
summarizer = LlmAgent(
    instruction=''
    Create summary in format: {user:preferred_format}

    If no preference set, use bullet points.
    ''
)
```





Reading app configuration:

```
Python
# Read global config in instruction
processor = LlmAgent(
    instruction=''
    Process with max depth: {app:max_depth}

    Default to 3 if not configured.
    ...
)
```

Core concept: State in parallel workflows

The aggregation pattern revisited

From part 1, you learned distinct keys for parallel agents. Here's the complete state flow:

```
Python
# Parallel agents write to distinct keys
researcher_a = LlmAgent(
    output_key='findings_academic', # → state['findings_academic']
    instruction='Research from academic sources'
)

researcher_b = LlmAgent(
    output_key='findings_news', # → state['findings_news']
    instruction='Research from news sources'
)

parallel = ParallelAgent(sub_agents=[researcher_a, researcher_b])

# Aggregator reads all keys
aggregator = LlmAgent(
    output_key='combined_findings', # → state['combined_findings']
    instruction=''
    Combine findings:
    - Academic: {findings_academic}
    - News: {findings_news}

    Create unified summary.
    ...
)

# Sequential ensures proper order
workflow = SequentialAgent(sub_agents=[parallel, aggregator])
```



State at each stage:

Stage	State Contains
After parallel	findings_academic, findings_news
After aggregator	All above + combined_findings

This diagram shows how state flows through the workflow, with each agent reading from and writing to shared session state:

```
None
graph TD
  subgraph "State Flow"
    INPUT[User Input] --> INTAKE[Intake Agent]
    INTAKE -->|"document_content"| STATE[(Session State)]

    STATE --> TOPIC[Topic Analyzer]
    STATE --> SENTIMENT[Sentiment Analyzer]

    TOPIC -->|"topics"| STATE
    SENTIMENT -->|"sentiment"| STATE

    STATE --> SUMMARIZER[Summarizer]
    STATE -->|"user:summary_format"| SUMMARIZER
    SUMMARIZER -->|"summary"| STATE
  end

  style STATE fill:#fff3cd
```

State flows through workflow stages, with each agent contributing to a shared session state.

